Strings

Introduction to Computer Programming

Chandra Gummaluru

chandra-gummaluru.github.io

A **string** (type `str`) is a sequence of characters, such as a word or a sentence. It starts and ends with a single quote `'` or a double quote `"`.

```
>>> 'Marcia'
'Marcia'
>>> "What a beautiful day!"
'What a beautiful day!'
>>> "It's 4 o'clock"
"It's 4 o'clock"
>>> ''
''
```

Use double quotes when the text contains a single quote. The **empty string** `''` has no characters at all.

Two operators work on strings: **concatenation** joins strings, and **repetition** repeats one.

concatenation +

two strings

```
>>> 'Mar' + 'cia'
'Marcia'
>>> 'Ha' + 2
TypeError
```

repetition *

a string and an `int`

```
>>> 'Ha' * 3
'HaHaHa'
>>> 2 * 'Ha'
'HaHa'
```

Both sides of + must be strings, and precedence is the same as for numbers.

What value does this expression produce?

```
1  'ab' + 'c' * 2
```

- ✓ `'abcc'`
- B `'abcabc'`
- C `'abc2'`
- D `TypeError`

The built-in function `len` gives the number of characters in a string. Spaces and punctuation count too.

```
>>> len('Marcia')
6
>>> len('Ha Ha')
5
>>> len('')
0
```

What value does this expression produce?

```
1  len('hello world')
```

Answer: `11`

`str`, `int` and `float` convert a value to that type. This is how a number and a string can be joined.

```
>>> str(108)
'108'
>>> 'CSC' + str(108)
'CSC108'
>>> int('42') + 1
43
>>> float('2.5')
2.5
```

`'4'` and `4` are different values: one is a string, the other an `int`.

What value does this expression produce?

```
1  str(4) + '2'
```

A 6

✓ '42'

C 42

D TypeError

strings.py

Program

```
1
```

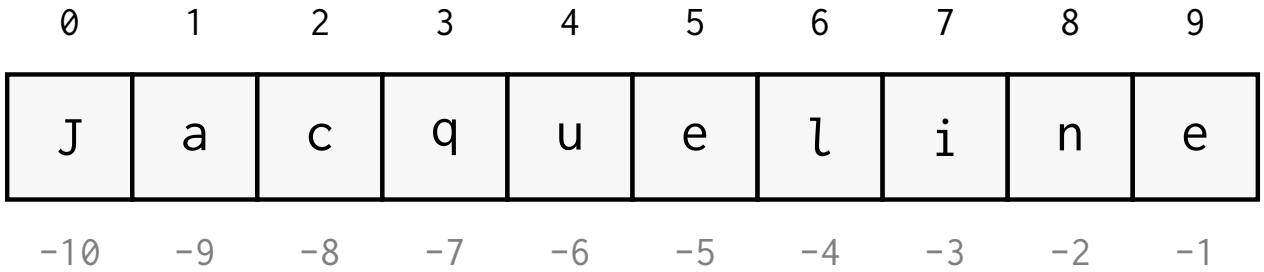
Shell

```
>>> 'Mar' + 'cia'
'Marcia'
>>> 'Ha' * 3
'HaHaHa'
>>> len('Marcia')
6
>>> 'CSC' + str(108)
'CSC108'
>>> 'Ha' + 2
```

```
Traceback (most recent call last):
  File "<shell>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
```

indexing and slicing

Each character in a string has a position, called its **index**. Indices start at `0` on the left; negative indices count from `-1` on the right.




```
s = 'Jacqueline'
```

`s[0]`	`'J'`
`s[4]`	`'u'`
`s[-1]`	`'e'`
`len(s)`	`10`

The last index is `len(s) - 1`, so `s[10]` is an `IndexError`.

What value does this expression produce?

```
1  'computer'[-3]
```

A 'p'

✓ 't'

C 'e'

D 'u'

Which expression produces `'LOL'`?

```
1  phrase = 'Laughing Out Loud'
```

✓ `phrase[0] + phrase[9] + phrase[-4]`

B `phrase[1] + phrase[10] + phrase[-4]`

C `phrase[0] + phrase[9] + phrase[-3]`

D `phrase[0] + phrase[8] + phrase[-5]`

A **slice** `s[start:end]` is the substring from index `start` up to, but not including, index `end`.

slice	value	
`s[1:4]`	`'acq'`	indices 1, 2, 3
`s[0:5]`	`'Jacqu'`	the first 5 characters
`s[:5]`	`'Jacqu'`	omit start: begin at 0
`s[1:len(s)]`	`'acqueline'`	everything but the first
`s[1:]`	`'acqueline'`	omit end: go to the end
`s[-4:]`	`'line'`	the last 4 characters

Here, `s = 'Jacqueline'`. The length of `s[a:b]` is `b - a`.

What value does this expression produce?

```
1  'computer'[3:6]
```

A `'pu'`

✓ `'put'`

C `'pute'`

D `'mpu'`

What value does `slice1` refer to?

```
1  phrase = 'big orange cat'
2  slice1 = phrase[:3]
```

✓ `'big'`

B `'big '`

C `'bi'`

D `'g'`

What value does `slice2` refer to?

```
1  phrase = 'big orange cat'
2  slice2 = phrase[-4:]
```

A `'cat'`

✓ `' cat'`

C `'e cat'`

D `' ca'`

What value does `slice3` refer to?

```
1  phrase = 'big orange cat'
2  slice3 = phrase[3:8]
```

A `'orang'`

B `' orang'`

✓ `' oran'`

D `'oran'`

An optional third number in a slice is the **stride**: the step between characters. A negative stride walks backwards.

slice	value	
`s[::2]`	`'Jculn'`	every other character, from the start
`s[1:8:3]`	`'aui'`	indices 1, 4, 7
`'stressed'[::-1]`	`'desserts'`	the whole string, reversed

In `s[start:end:stride]`, any of the three can be left out.

stride.ipynb

You know that the slicing operation `s[1:4]` will produce the string `'acq'`. The slicing operation has an optional third parameter that determines the *stride* (or distance between characters) in the slice. For example, the slicing operation `s[::2]` will produce the string `'Jculn'`, which has every other character in `'Jacqueline'`, starting from the first character in the string, and up to the end of the string. Use a negative stride to work backwards through a string.

In [ ]
```
s = 'Jacqueline'
```

(a) Write an expression that uses slicing on `s` to produce the string `'aqeie'`.

In [2]
```
# (a)
s[1::2]
```

```
'aqeie'
```

(b) Write an expression that uses slicing on `s` to produce the string `'enileuqcaJ'`.

In [3]

```
# (b)
s[::-1]
```

```
'enileuqcaJ'
```

(c) Write an expression that uses slicing on `s` to produce the string `'eieqa'`.

In [4]

```
# (c)
s[::-2]
```

```
'eieqa'
```

string methods

A **method** is a function that belongs to a type. We call a method on a value using **dot notation**.

```
value.method_name(arguments)
```

```
>>> s = 'Hello'
>>> s.upper()
'HELLO'
>>> s.count('l')
2
>>> s
'Hello'
```

The value before the dot is passed to the method automatically, and any other arguments go in the parentheses. String methods return a new value; s itself never changes.

Some common string methods:

method	**example**	**what it produces**
`upper`, `lower`	`'Hi'.lower()`	a copy in upper or lower case
`count`	`'banana'.count('a')`	the number of occurrences
`find`	`'banana'.find('n')`	the first index, or `-1`
`replace`	`'banana'.replace('a', 'o')`	a copy with matches replaced
`strip`	`' hi '.strip()`	a copy without outer spaces

`dir(str)` lists every string method, and `help(str.find)` explains one of them.

string_methods.py

Program

```
1
```

Shell

```
>>> s = 'Hello'
>>> s.upper()
'HELLO'
>>> s.count('l')
2
>>> s.find('z')
-1
>>> s.replace('l', 'L')
'HeLLo'
>>> '  hi  '.strip()
'hi'
```